

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

**Отчёт
Лабораторная работа №1**

Выполнил:
Ермаков Максим Олегович
К3340

Проверил:
Добряков Д. И.

Санкт-Петербург
2025 г.

Задача

В рамках лабораторной работы №1 требовалось реализовать полноценный REST API бэкенд-приложения на основе готового boilerplate-проекта. В качестве предметной области было выбрано приложение для бронирования столиков в ресторанах.

За основу был взят проект `express-typeorm-boilerplate`, реализующий минимальную структуру приложения на Node.js с использованием Express и TypeORM. Исходный boilerplate необходимо было адаптировать и расширить под конкретную предметную область: добавить доменные сущности, бизнес-логику, разграничение доступа и документацию.

Функциональность приложения по варианту включает:

- регистрацию и вход в систему;
- личный кабинет пользователя;
- поиск ресторанов с фильтрацией по кухне, расположению и ценовой категории;
- страницу ресторана с меню, фотографиями и отзывами;
- создание и управление бронированиями;
- историю бронирований пользователя.

Ход работы

1. Изучение исходного boilerplate

На первом этапе был детально изучен проект `express-typeorm-boilerplate`. Boilerplate предоставляет базовую структуру директорий, конфигурацию TypeScript, подключение к базе данных через TypeORM, а также пример простого контроллера. В рамках изучения были проанализированы: способ регистрации маршрутов через библиотеку `routing-controllers`, механизм инициализации соединения с PostgreSQL, а также принятые соглашения по именованию и размещению файлов.

2. Анализ материалов из предыдущих домашних заданий

Перед началом разработки были проанализированы материалы, подготовленные в рамках предыдущих домашних заданий. Из ДЗ1 была использована ER-диаграмма и описание модели базы данных, определявшие набор сущностей и связей между ними. Из ДЗ2 — контракт API, спецификация в формате TypeSpec и сгенерированное OpenAPI-описание, задавшие структуру эндпоинтов, форматы запросов и ответов. Эти материалы послужили проектной основой для реализации.

3. Реализация доменной модели базы данных

На основе ER-диаграммы были реализованы TypeORM-сущности для всех объектов предметной области. Итоговый набор сущностей:

- User — пользователь системы с ролью USER или ADMIN;
- Location — географическое расположение ресторана;
- Cuisine — тип кухни (справочник);
- Restaurant — ресторан с ценовой категорией (LOW, MEDIUM, HIGH) и статусом публикации;
- RestaurantTable — столик ресторана с указанием вместимости;
- RestaurantPhoto — фотография ресторана;
- MenuCategory — категория меню;
- MenuItem — позиция меню в рамках категории;
- Review — отзыв пользователя о ресторане;

- Reservation — бронирование столика со статусами PENDING, CONFIRMED, CANCELLED, COMPLETED.

Между сущностями были настроены необходимые связи: пользователь имеет множество бронирований и отзывов; ресторан привязан к одной локации, связан со многими кухнями и содержит столики, фотографии, категории меню, отзывы и бронирования; категория меню включает набор блюд; бронирование связано с пользователем, рестораном и конкретным столиком; отзыв привязан к пользователю и ресторану.

4. Реализация контроллеров и REST-маршрутов

Бизнес-логика приложения организована в шести контроллерах:

- AuthController — регистрация нового пользователя, вход по email и паролю, получение JWT access token;
- UsersController — получение и обновление профиля текущего пользователя;
- ReferenceController — получение справочников (локации, кухни) для использования в фильтрах и формах;
- RestaurantsController — поиск ресторанов с фильтрацией по кухне, локации и ценовой категории; просмотр детальной информации о ресторане, его меню, фотографий, отзывов; проверка доступности столиков по дате, времени и количеству гостей;
- ReservationsController — создание бронирования, просмотр списка своих бронирований, просмотр бронирования по идентификатору, обновление и отмена бронирования; создание и редактирование отзыва;
- AdminController — создание и редактирование локаций, создание и публикация ресторанов, управление столиками, категориями меню, блюдами и фотографиями, изменение статуса бронирования.

5. Авторизация и разграничение прав доступа

Аутентификация реализована на основе JWT. После успешного входа клиент получает access token, который передаётся в

заголовке `Authorization: Bearer <token>` при последующих запросах. Для проверки токена подключён соответствующий `middleware`, который также извлекает информацию о текущем пользователе.

В системе предусмотрены две роли с разным уровнем доступа:

- **USER** — просмотр ресторанов, создание бронирований, просмотр своей истории бронирований, оставление отзывов; редактирование только собственных данных, бронирований и отзывов;
- **ADMIN** — все права пользователя плюс создание и редактирование локаций, ресторанов, столиков, меню, фотографий; публикация ресторанов; изменение статуса любого бронирования.

Для ограничения доступа к административным маршрутам реализован отдельный `middleware` проверки роли администратора. В процессе работы была выявлена и устранена ошибка порядка подключения `middleware`: изначально `middleware` аутентификации и проверки роли применялись в неверном порядке, из-за чего даже авторизованный администратор получал ошибку доступа. После корректировки порядка подключения `middleware` проблема была решена.

6. Реализация инфраструктурных компонентов

Помимо основной бизнес-логики, были реализованы следующие инфраструктурные компоненты:

- единый формат ошибок API — все ошибки возвращаются в унифицированной структуре JSON;
- валидация входных данных на основе библиотеки `class-validator`;
- сериализация ответов API — чувствительные поля (например, хэш пароля) исключаются из ответа;
- пагинация для списочных эндпоинтов;
- автоматическое создание стартовых данных при первом запуске: создаётся учётная запись администратора и базовый набор кухонь;
- Swagger-документация (Swagger UI) для просмотра спецификации и ручного вызова эндпоинтов.

При запуске PostgreSQL в Docker возникли проблемы с версией образа и директорией данных. Конфигурация Docker-контейнера была скорректирована, после чего база данных запустилась корректно.

7. Тестирование API через Swagger

Для проверки работоспособности реализованных эндпоинтов в первую очередь использовался Swagger UI. В процессе ручного тестирования были проверены следующие сценарии:

- регистрация нового пользователя и вход в систему;
- получение профиля с авторизацией через Bearer token;
- создание локации администратором;
- создание ресторана и его публикация;
- добавление столиков, категорий меню с позициями и фотографий;
- поиск ресторана по фильтрам;
- проверка доступности столиков;
- создание бронирования и просмотр истории бронирований;
- добавление отзыва;
- подтверждение бронирования администратором.

В ходе тестирования через Swagger была обнаружена проблема: документация не всегда корректно подставляла Bearer token в защищённые методы. Конфигурация OpenAPI-описания для защищённых маршрутов была доработана, после чего авторизация в Swagger начала работать корректно.

Вывод

В ходе лабораторной работы был реализован полноценный REST API для приложения бронирования столиков в ресторанах. Цель работы достигнута: минимальный boilerplate на Express + TypeORM был адаптирован под конкретную предметную область и расширен до готового бэкенд-приложения.

В рамках работы была спроектирована и реализована доменная модель из десяти сущностей (User, Location, Cuisine, Restaurant, RestaurantTable, RestaurantPhoto, MenuCategory, MenuItem, Review, Reservation) с необходимыми связями и перечислениями. На основе контракта API из предыдущего домашнего задания реализованы шесть контроллеров, обеспечивающих как пользовательские сценарии (регистрация, поиск ресторанов, бронирование, отзывы), так и административные функции (управление контентом, публикация, изменение статусов).

Реализована JWT-аутентификация с разграничением прав доступа между ролями USER и ADMIN. Подключена валидация входных данных, унифицированный формат ошибок, сериализация ответов и пагинация. Настроена Swagger-документация, позволяющая просматривать спецификацию и вызывать эндпоинты непосредственно из браузера.

Работоспособность API подтверждена ручным тестированием через Swagger UI: были последовательно проверены все ключевые сценарии — от регистрации и авторизации до создания бронирования и управления контентом администратором. В ходе работы были выявлены и устранены две практические проблемы: некорректная передача Bearer token в Swagger и ошибка порядка middleware в административных маршрутах.

Лабораторная работа позволила получить практический опыт разработки структурированного REST API: от изучения boilerplate и проектирования доменной модели до реализации контроллеров, настройки авторизации и организации комплексного тестирования.